CHAPTER 15

CloudFormation and CDK

Infrastructure as Code · CDK Stacks · Constructs · Deploy · Testing · ShopFlow

15-01 Why Infrastructure as Code?

■ Infrastructure as Code (IaC)

Infrastructure as Code means defining your AWS resources (VPCs, ECS clusters, databases, load balancers) in code files — then deploying them by running a command. Instead of clicking through the AWS Console every time, you write code once and deploy it reliably to dev, staging, and production. AWS CDK and CloudFormation are the two primary IaC tools for AWS.

Example: ShopFlow's entire AWS environment — VPC, ALB, ECS cluster, Aurora PostgreSQL, DynamoDB, ElastiCache, OpenSearch, S3, CloudFront, Route 53, WAF, CodePipeline — is defined in CDK code. Deploying a new environment (e.g., for a new customer region) takes 25 minutes and one command: cdk deploy --all.

Without IaC (manual)	With IaC (CDK/CloudFormation)
Click through Console for each resource (error-prone, slow)	Write code once; deploy with one command
Dev environment differs from prod (config drift)	Dev, staging, prod are identical (same code, different parameters)
No record of what changed or why	Git history tracks every change with author and reason
Difficult to recreate after disaster	Redeploy to new region in 25 minutes with same cdk deploy command
Hard to review changes before applying	cdk diff shows exactly what will change before deployment
Environment setup takes days of manual work	New developer sets up local env with npm install and cdk deploy

15-02 CloudFormation vs CDK — Understanding the Relationship

CloudFormation is the AWS native IaC service. CDK is a higher-level tool that generates CloudFormation templates from code. You write CDK in Java, Python, or TypeScript — CDK compiles it to a CloudFormation template — CloudFormation deploys the resources.

Aspect	CloudFormation	AWS CDK
What you write	YAML or JSON template (verbose, 500+ lines for ShopFlow)	Java/Python/TypeScript code using familiar OOP patterns
Level of abstraction	Low: you specify every resource property explicitly	High: L2 constructs provide sensible defaults; you only co
Reusability	Copy-paste YAML; limited native reuse	OOP constructs, inheritance, loops, functions — real code
What runs the deployment	CloudFormation service directly	CDK synthesizes CloudFormation template; CloudFormat
Rollback on failure	Automatic rollback if any resource fails	Automatic rollback (via CloudFormation underneath)
ShopFlow choice	Not used directly (CDK generates the templates)	ShopFlow uses CDK (Java) for all infrastructure

15-03 CDK Project Structure — ShopFlow Layout

A CDK app is a Java Maven project. ShopFlow organizes CDK code into separate stacks for networking, data, services, and pipeline — each can be deployed independently.

```
// ShopFlow CDK project structure shopflow-cdk/ ■■■ pom.xml // Maven build file; CDK
dependencies ■■■ cdk.json // CDK app entry point and context variables ■■■
src/main/java/com/shopflow/cdk/ ■ ■■■ ShopFlowApp.java // Main entry: creates all stacks ■
```

```
■■■ stacks/ ■ ■ ■■■ NetworkStack.java // VPC, subnets, NAT Gateway, security groups ■ ■ ■■■ DataStack.java // Aurora, DynamoDB, ElastiCache, OpenSearch ■ ■ ■■■ ServicesStack.java // ECS cluster, Fargate services, ALB, ECR ■ ■ ■■■ TrafficStack.java // Route 53, ACM, WAF, CloudFront ■ ■ ■■■ ObservabilityStack.java // CloudWatch alarms, dashboards, X-Ray ■ ■ ■■■ PipelineStack.java // CodePipeline CI/CD for ShopFlow ■ ■■■ constructs/ ■ ■■■ ShopFlowService.java // Reusable construct: ECS service + ALB + monitoring ■ ■■■ AuroraCluster.java // Reusable construct: Aurora cluster with replicas ■ ■■■ ShopFlowVpc.java // Reusable construct: VPC with standard ShopFlow config // cdk.json: app entry point { "app": "mvn -e -q compile exec:java", "context": { "env": "prod", "region": "us-east-1", "account": "123456789012" } }
```

15-04 Your First CDK Stack — ShopFlow Network

The NetworkStack creates the VPC, subnets, and security groups that all other ShopFlow stacks depend on. This is always the first stack to deploy.

```
// NetworkStack.java — ShopFlow VPC and security groups public class NetworkStack extends Stack { private final Vpc vpc; private final SecurityGroup albSg; private final SecurityGroup ecsSg; private final SecurityGroup dbSg; public NetworkStack(App app, String id, StackProps props) { super(app, id, props); // Create VPC: 2 AZs, public + private + isolated subnets vpc = Vpc.Builder.create(this, "ShopFlowVPC") .vpcName("shopflow-prod") .maxAzs(2) .natGateways(2) // one per AZ for HA .subnetConfiguration(List.of( SubnetConfiguration.builder() .name("Public").subnetType(SubnetType.PUBLIC) .cidrMask(24).build(), // 10.0.0.0/24, 10.0.1.0/24 SubnetConfiguration.builder() .name("Private").subnetType(SubnetType.PRIVATE_WITH_EGRESS) .cidrMask(24).build(), // 10.0.2.0/24, 10.0.3.0/24 (ECS tasks) SubnetConfiguration.builder() .name("Isolated").subnetType(SubnetType.PRIVATE_ISOLATED) .cidrMask(24).build())) // 10.0.4.0/24, 10.0.5.0/24 (databases) .build(); // ALB security group: allow 443 from internet albSg = SecurityGroup.Builder.create(this, "AlbSG") .vpc(vpc).securityGroupName("shopflow-alb-sg").build(); albSg.addIngressRule(Peer.anyIpv4(), Port.tcp(443), "HTTPS from internet"); albSg.addIngressRule(Peer.anyIpv4(), Port.tcp(80), "HTTP redirect"); // ECS security group: allow 8080 from ALB only ecsSg = SecurityGroup.Builder.create(this, "EcsSG") .vpc(vpc).securityGroupName("shopflow-ecs-sg").build(); ecsSg.addIngressRule(albSg, Port.tcp(8080), "From ALB only"); // DB security group: allow 5432 from ECS only dbSg = SecurityGroup.Builder.create(this, "DbSG") .vpc(vpc).securityGroupName("shopflow-db-sg").build(); dbSg.addIngressRule(ecsSg, Port.tcp(5432), "Aurora from ECS"); dbSg.addIngressRule(ecsSg, Port.tcp(6379), "Redis from ECS"); } // Getters so other stacks can reference these resources public Vpc getVpc() { return vpc; } public SecurityGroup getEcsSg() { return ecsSg; } public SecurityGroup getDbSg() { return dbSg; } }
```

15-05 CloudFormation Console — Viewing Your Stack

When CDK deploys a stack, CloudFormation creates and manages a "stack" in the CloudFormation Console. You can view the status of each resource deployment here.

■ View CDK Stack in CloudFormation Console
1. AWS Console -> CloudFormation -> Stacks
2. Find "ShopFlowNetworkStack" in the list
3. Click the stack name -> opens Stack Detail
4. Click "Events" tab -> shows each resource being created/updated in real-time
5. Click "Resources" tab -> lists all resources with their logical ID, type, and status
6. Click "Outputs" tab -> shows values exported from this stack (VPC ID, subnet IDs)
7. Click "Template" tab -> shows the CloudFormation YAML that CDK generated
8. To see what cdk deploy would change without deploying: run "cdk diff" in terminal

■ aws › CloudFormation > Stacks > ShopFlowNetworkStack

Stack: ShopFlowNetworkStack

■ Stack Info

■ Status: CREATE_COMPLETE | Created: 2026-06-01 14:32:15 UTC

■ Description: ShopFlow Production VPC and Security Groups

■ Resources (14 resources)

■ AWS::EC2::VPC | ShopFlowVPC | CREATE_COMPLETE

■ AWS::EC2::Subnet | PrivateSubnet1 | CREATE_COMPLETE

■ AWS::EC2::Subnet | PrivateSubnet2 | CREATE_COMPLETE

■ AWS::EC2::NatGateway | NATGateway1 | CREATE_COMPLETE

■ AWS::EC2::NatGateway | NATGateway2 | CREATE_COMPLETE

■ AWS::EC2::SecurityGroup | AlbSG | CREATE_COMPLETE

■ AWS::EC2::SecurityGroup | EcsSG | CREATE_COMPLETE

■ AWS::EC2::SecurityGroup | DbSG | CREATE_COMPLETE

■ Outputs

■ VpcId: vpc-0abc123def456 | Exported as: ShopFlowNetworkStack-VpcId

■ PrivateSubnetIds: subnet-0abc,subnet-0def | Exported as: ShopFlowNetworkStack-PrivateSubnets

---

15-06 CDK Constructs — L1, L2, and L3

CDK has three levels of constructs. L1 maps directly to CloudFormation resources. L2 adds smart defaults and helper methods. L3 (patterns) combine multiple resources into a complete solution.

Level	Name	Description	ShopFlow Example
L1	Cfn* (CloudFormation)	Direct mapping to one CloudFormation resource; no defaults, pure CFN	CfnWebACL for WAF (no L2 equivalent yet); CfnS
L2	Standard	One resource with sensible defaults; helper methods; most CDK classes are L2	ApplicationLoadBalancedFargateService, Vpc, B
L3	Patterns	Multiple resources combined; end-to-end solution in one construct	ApplicationLoadBalancedFargateService — creat

```
// L3 Pattern: deploy ECS service + ALB in one construct // (under the hood this creates: ECS TaskDef, FargateService, ALB, Listener, TargetGroup, LogGroup) ApplicationLoadBalancedFargateService shopflowProductSvc = ApplicationLoadBalancedFargateService.Builder.create(this, "ProductSvc") .cluster(cluster) .cpu(512) .memoryLimitMiB(1024) .desiredCount(2) .taskImageOptions(ApplicationLoadBalancedTaskImageOptions.builder() .image(ContainerImage.fromEcrRepository(productSvcRepo, "latest")) .containerPort(8080) .build()) .publicLoadBalancer(true) .build(); // vs L2 approach: 60+ lines to create all the same resources manually // Use L3 patterns for quick setup; switch to L2 when you need fine-grained control
```

15-07 CDK CLI Commands — Bootstrap, Synth, Diff, Deploy

The CDK CLI is how you interact with your CDK app from the terminal. These are the commands ShopFlow engineers use every day.

Command	What It Does	When to Use
cdk bootstrap	Creates CDK staging resources in your AWS account: S3 bucket for assets, ECR repo for Docker images, IAM roles	First time you use CDK in an account/region
cdk synth	Synthesizes CloudFormation templates from CDK code. Output is JSON/YAML. Does NOT deploy	To inspect what CloudFormation will create
cdk diff	Shows what will CHANGE if you deploy — new resources, modified properties, deleted resources	Always run before deploy to verify changes
cdk deploy	Deploys the stack to AWS. Runs synth, uploads assets to S3, triggers CloudFormation	Normal deployment workflow
cdk deploy --all	Deploys ALL stacks in the CDK app in dependency order.	Full environment deployment; ShopFlow uses
cdk destroy	Deletes the stack and all its resources. IRREVERSIBLE.	Cleanup dev/test environments; never run on
cdk list	Lists all stacks in the CDK app	See all stacks and their dependencies

15-08 Stack Dependencies — Deployment Order

When one CDK stack uses resources from another (e.g., ServicesStack uses the VPC from NetworkStack), CDK automatically detects the dependency and deploys stacks in the right order.

```
// ShopFlowApp.java: define stacks and dependencies public class ShopFlowApp { public static void main(String[] args) { App app = new App(); // Stack 1: Network (no dependencies) NetworkStack network = new NetworkStack(app, "ShopFlowNetworkStack", StackProps.builder().env(env).build()); // Stack 2: Data (depends on Network for VPC + security groups) DataStack data = new DataStack(app, "ShopFlowDataStack", DataStackProps.builder() .vpc(network.getVpc()) // cross-stack reference .dbSg(network.getDbSg()) // CDK automatically adds stack dependency .build()); // Stack 3: Services (depends on Network + Data) ServicesStack services = new ServicesStack(app, "ShopFlowServicesStack", ServicesStackProps.builder() .vpc(network.getVpc()) .ecsSg(network.getEcsSg()) .auroraCluster(data.getAuroraCluster()) // cross-stack reference .redisCluster(data.getRedisCluster()) .build()); // cdk deploy --all deploys in order: Network -> Data -> Services // If Network fails, Data and Services are not attempted app.synth(); } } // Cross-stack reference becomes a CloudFormation Export/ImportValue: // NetworkStack exports: VpcId, PrivateSubnets, EcsSgId // DataStack imports: VpcId (to place Aurora in the correct VPC) // CDK handles this automatically — you just pass the object
```

15-09 CDK Aspects — Enforce Standards Across All Stacks

CDK Aspects let you apply policies to every resource in your app. ShopFlow uses Aspects to enforce tagging, encryption, and deletion protection across all resources — no matter which engineer writes the CDK code.

```
// CDK Aspect: enforce required tags on every resource public class TaggingAspect implements IAspect { @Override public void visit(IConstruct node) { if (node instanceof CfnResource) { Tags.of(node).add("Project", "ShopFlow"); Tags.of(node).add("Owner", "platform-team"); Tags.of(node).add("CostCenter", "ecommerce"); // Tag value comes from CDK context (different per environment) String env = (String) node.getNode().tryGetContext("env"); Tags.of(node).add("Environment", env != null ? env : "dev"); } } } // CDK Aspect: require encryption on all S3 buckets public class S3EncryptionAspect implements IAspect { @Override public void visit(IConstruct node) { if (node instanceof CfnBucket) { CfnBucket bucket = (CfnBucket) node; if (bucket.getBucketEncryption() == null) { // Fail the synthesis if any bucket is created without encryption Annotations.of(node).addError( "S3 bucket must have encryption enabled. " + "Add .encryption(BucketEncryption.S3_MANAGED) to Bucket.Builder"); } } } } // Apply aspects to the entire app (affects ALL stacks) Aspects.of(app).add(new TaggingAspect()); Aspects.of(app).add(new S3EncryptionAspect());
```

15-10 CloudFormation Stack Operations — Update and Rollback

CloudFormation performs a Change Set calculation before modifying resources. Some changes are safe (update in-place); others require replacement (delete old, create new). Knowing which is which prevents outages.

Resource	Safe Update (in-place)	Replacement Required (delete + recreate)
ECS Task Definition	Update container image tag, CPU/memory, env va	Cannot replace task def — create new revision (ECS does this

Resource	Safe Update (in-place)	Replacement Required (delete + recreate)
Aurora Parameter Group	Most parameter changes	Changing engine version (major version upgrade = replacemen
DynamoDB Table	Add GSI, change billing mode, update TTL	Change partition key, sort key, or table name — forces full repla
S3 Bucket	Change lifecycle rules, versioning, CORS	Change bucket name — replacement (new bucket is empty)
EC2 Instance Type	Stop, change type, start	Replacement for some types (depends on hypervisor compatib
VPC Subnet CIDR	Not possible without replacement	Changing CIDR -> replacement -> ALL resources in subnet del
ALB Listener Port	Update in-place	Change load balancer type (ALB -> NLB) -> replacement

WARN Database Replacement = DATA LOSS

If CloudFormation marks a database resource (Aurora, DynamoDB table) as requiring REPLACEMENT, it will DELETE the old database and CREATE a new empty one. Always run cdk diff before deploying data stack changes. If you see "AWS::RDS::DBCluster | Replace: true", stop and find a different migration approach. Never blindly run cdk deploy on data stacks without checking the diff first.

15-11 CloudFormation Drift Detection

Drift occurs when someone manually modifies a resource in the AWS Console instead of updating the CDK code. CloudFormation Drift Detection finds these differences so you can fix them.

■ Detect Drift in ShopFlow ServicesStack
1. CloudFormation -> Stacks -> ShopFlowServicesStack
2. Click "Stack actions" -> "Detect drift"
3. Wait ~2 minutes while CloudFormation checks all resources
4. Click "Stack actions" -> "View drift results"
5. Drifted resources appear in red: shows expected vs actual property values
6. Example: someone manually changed ECS desired count from 6 to 2 in the console
7. Fix: update CDK code to match current state, OR redeploy CDK to revert the console change
8. Best practice: never manually change resources managed by CloudFormation/CDK

15-12 CDK Testing — Unit and Integration Tests

CDK constructs are Java code — they can be unit tested. ShopFlow tests CDK stacks to ensure security properties (encryption enabled, public access blocked) and configuration correctness before deploying.

```
// CDK Unit Test: verify NetworkStack creates VPC with correct config // pom.xml: add
aws-cdk-lib:test-utils, org.junit.jupiter:junit-jupiter class NetworkStackTest { @Test void
vpc_hasCorrectConfiguration() { App app = new App(); NetworkStack stack = new NetworkStack(app,
"TestStack", StackProps.builder().env(Environment.builder()
.account("123456789").region("us-east-1").build()).build()); Template template =
Template.fromStack(stack); // Assert VPC is created template.hasResource("AWS::EC2::VPC",
Match.objectLike( Map.of("Properties", Map.of( "EnableDnsHostnames", true, "EnableDnsSupport",
true)))); // Assert 2 NAT Gateways (for HA) template.resourceCountIs("AWS::EC2::NatGateway",
2); // Assert ALB SG allows port 443 from internet
template.hasResourceProperties("AWS::EC2::SecurityGroup", Match.objectLike(Map.of(
"GroupDescription", "shopflow-alb-sg", "SecurityGroupIngress", Match.arrayWith(List.of(
Match.objectLike(Map.of("FromPort", 443, "CidrIp", "0.0.0.0/0")))))); // Assert no public
database subnets (all DB subnets are ISOLATED) Map[String, Object] isolatedSubnets =
template.findResources("AWS::EC2::Subnet", Match.objectLike(Map.of("Properties",
Map.of("MapPublicIpOnLaunch", false)))); assert isolatedSubnets.size() >= 2 : "Expected at
```

```
least 2 isolated subnets for databases"; } }
```

15-13 ShopFlow CDK Stack Architecture

Here is the complete CDK stack architecture showing how ShopFlow's stacks depend on each other and what each contains.


ShopFlow CDK App (cdk deploy --all)
NetworkStack (deploys first)
VPC — 2 AZs, public + private + isolated subnets
NAT Gateways — 2x (one per AZ for HA)
Security Groups — ALB, ECS, Aurora/Redis, OpenSearch
Route 53 Private Hosted Zone — shopflow.internal
DataStack (depends on: NetworkStack)
Aurora PostgreSQL 15 — 1 writer + 2 readers, Multi-AZ
DynamoDB — orders, cart, sessions tables
ElastiCache Redis — cluster mode, 3 shards
Amazon OpenSearch — 3-node domain with UltraWarm
ServicesStack (depends on: Network + Data)
ECS Cluster — shopflow-prod with Container Insights
Fargate Services — product-svc, order-svc, cart-svc, user-svc
ALB — internet-facing, HTTPS, WAF integrated
ECR Repositories — one per service
TrafficStack (depends on: ServicesStack)
Route 53 — A records, health checks, failover routing
ACM Certificate — *.shopflow.com (DNS-validated)
CloudFront Distribution — /static/* CDN caching
ObservabilityStack (depends on: ServicesStack)
CloudWatch Alarms — 12 alarms across all services
CloudWatch Dashboard — ShopFlow-Production-Ops
X-Ray Sampling Rules — 5% default, 100% checkout
PipelineStack (depends on: all stacks)
CodePipeline — source -> build -> test -> deploy
CodeBuild — Maven build + Docker build + CDK deploy

In this lab you will install the CDK CLI, create a new CDK app, and deploy a simple S3 bucket with versioning enabled. Time estimate: 30 minutes.

■ Lab: Create and Deploy a CDK Stack
1. Install CDK CLI: npm install -g aws-cdk (requires Node.js 18+)
2. Verify: cdk --version (should show 2.x.x)
3. Create new project: mkdir shopflow-lab && cd shopflow-lab && cdk init app --language java
4. Open pom.xml: verify software.amazon.awscdk dependency is present
5. Open src/main/java/.../ShopflowLabStack.java
6. Add S3 bucket with versioning: Bucket.Builder.create(this,"LabBucket").versioned(true).build()
7. Synthesize: cdk synth (outputs CloudFormation YAML to cdk.out/)
8. Bootstrap (once per account): cdk bootstrap aws://YOUR-ACCOUNT-ID/us-east-1
9. Deploy: cdk deploy (review changes, confirm with y)
10. Verify: AWS Console -> S3 -> find bucket named "shopflowlab-labbucket-XXXX"
11. View in CloudFormation: CloudFormation -> Stacks -> ShopflowLabStack -> Resources tab
12. Cleanup: cdk destroy (confirm with y — deletes the S3 bucket)

15-15 CloudFormation StackSets — Multi-Account Deployment

StackSets deploy the same CloudFormation template to multiple AWS accounts and regions simultaneously. ShopFlow uses StackSets for security baseline configuration (CloudTrail, GuardDuty, IAM password policy) that must be consistent across all accounts.

Use Case	StackSets or Not?	Why
Deploy ShopFlow ECS to us-east-1 and eu-west-1	No — CDK deploy	Single account, different region — CDK handles this with env para
Enforce CloudTrail in all 5 ShopFlow AWS accounts	YES — StackSets	Same account, multiple regions; StackSets deploy and maintain the stack in all
Enable GuardDuty in every account and region	YES — StackSets	Security baseline; must apply uniformly; StackSets with auto-depl
Deploy ShopFlow product service	No — CodePipeline + CDK	Application deployments use CI/CD, not StackSets

15-16 CDK Nested Stacks — Breaking Up Large Stacks

CloudFormation has a limit of 500 resources per stack. A complex ShopFlow deployment with all services could exceed this. Nested Stacks let you split a large stack into child stacks while keeping them logically grouped.

```
// CDK: NestedStack for ECS services (keeps resource count manageable) public class
EcsServicesNestedStack extends NestedStack { public EcsServicesNestedStack(Construct scope,
String id, Vpc vpc, SecurityGroup ecsSg) { super(scope, id); // ECS Cluster + 4 services here
Cluster cluster = Cluster.Builder.create(this, "Cluster") .clusterName("shopflow-prod")
.vpc(vpc) .containerInsights(true) .build(); // product-svc, order-svc, cart-svc, user-svc each
add ~15-20 resources // Nested stack isolates these from the parent stack's resource count
createFargateService("product-svc", cluster, ecsSg, ...); createFargateService("order-svc",
cluster, ecsSg, ...); } } // Parent ServicesStack instantiates nested stack public class
ServicesStack extends Stack { public ServicesStack(App app, String id, ...) { super(app, id,
props); // Nested stack: deploys as a child CloudFormation stack new
EcsServicesNestedStack(this, "EcsServices", network.getVpc(), network.getEcsSg()); new
DataNestedStack(this, "DataServices", network.getVpc(), network.getDbSg()); } }
```

15-17 CDK Best Practices Summary

Practice	Details
One stack per lifecycle	Resources that change together go in the same stack. Separate network (changes rarely) from serv
Use L2 constructs by default	L2 constructs have secure defaults (encryption, logging, IAM least-privilege). Only drop to L1 (Cfn*)
Always run cdk diff before deploy	Never run cdk deploy blindly. cdk diff shows exactly what will change. For data stacks, if you see "R
Parameterize with context, not hardcoding	Use cdk.json context for environment-specific values (account IDs, region, instance sizes). Never h
Use CDK Aspects for org-wide policy	Enforce tagging, encryption, and naming conventions with Aspects applied to the App level. This ca
Test CDK code with Template assertions	Use aws-cdk-lib/assertions to write unit tests that verify CloudFormation properties. Catch security r
Version-pin CDK dependencies	Pin aws-cdk-lib to a specific version in pom.xml. CDK minor versions can change L2 default behavi

15-18 Q&A and Cost — CloudFormation and CDK

Question	Answer
CloudFormation cost?	CloudFormation is FREE. You only pay for the AWS resources it creates (ECS tasks, RDS insta
How long does cdk deploy take for ShopFlow?	Full environment (--all stacks): 25-35 minutes (Aurora takes longest: 15-20 min). Services-only
Can I import existing resources (created via Console) into CDK?	Yes — use CloudFormation resource import or aws-cdk-lib's fromLookup methods. For example
What happens if cdk deploy fails halfway?	CloudFormation automatically rolls back to the previous state. Resources that were successfully
How do dev and prod environments differ in CDK?	CDK context (cdk.json or -c flag): dev uses smaller instance sizes (db.t3.medium), fewer ECS t

15-19 Reusable CDK Construct — ShopFlowFargateService

ShopFlow encapsulates ECS Fargate service creation into a reusable construct. Every microservice (product-svc, order-svc, cart-svc) uses this construct — ensuring consistent configuration, monitoring, and security without duplicating code.

```
// Reusable CDK Construct: one line deploys a complete ECS service public class
ShopFlowFargateService extends Construct { private final FargateService service; private final
ApplicationTargetGroup targetGroup; public ShopFlowFargateService(Construct scope, String id,
ShopFlowServiceProps props) { super(scope, id); // Task definition FargateTaskDefinition
taskDef = FargateTaskDefinition.Builder.create(this, "TaskDef") .cpu(props.getCpu())
.memoryLimitMiB(props.getMemoryMiB()) .taskRole(createTaskRole(props.getServiceName()))
.build(); // Container with CloudWatch logs + X-Ray sidecar taskDef.addContainer("app",
ContainerDefinitionOptions.builder() .image(ContainerImage.fromEcrRepository(props.getRepo(),
"latest")) .portMappings(List.of(PortMapping.builder().containerPort(8080).build()))
.logging(LogDrivers.awsLogs(AwsLogDriverProps.builder() .logGroup(new LogGroup(this, "Logs",
LogGroupProps.builder() .logGroupName("/ecs/shopflow/" + props.getServiceName())
.retention(RetentionDays.ONE_MONTH).build())) .streamPrefix(props.getServiceName()).build()))
.environment(Map.of( "SPRING_PROFILES_ACTIVE", "prod", "SERVICE_NAME", props.getServiceName()))
.secrets(props.getSecrets()) // from SSM Parameter Store .build()); // Fargate service service
= FargateService.Builder.create(this, "Service") .cluster(props.getCluster())
.taskDefinition(taskDef) .desiredCount(props.getDesiredCount())
.healthCheckGracePeriod(Duration.seconds(120)) .securityGroups(List.of(props.getEcsSg()))
.vpcSubnets(SubnetSelection.builder() .subnetType(SubnetType.PRIVATE_WITH_EGRESS).build())
.build(); // Auto scaling: scale on CPU ScalableTaskCount scaling = service.autoScaleTaskCount(
EnableScalingProps.builder().minCapacity(props.getMinCount())
.maxCapacity(props.getMaxCount()).build()); scaling.scaleOnCpuUtilization("CpuScaling",
CpuUtilizationScalingProps.builder() .targetUtilizationPercent(70)
.scaleInCooldown(Duration.seconds(300)) .scaleOutCooldown(Duration.seconds(60)) .build()); //
ALB target group targetGroup = ApplicationTargetGroup.Builder.create(this, "TG")
.vpc(props.getVpc()) .port(8080).protocol(ApplicationProtocol.HTTP) .targetType(TargetType.IP)
.healthCheck(HealthCheck.builder() .path("/actuator/health").healthyHttpCodes("200")
.interval(Duration.seconds(30)) .healthyThresholdCount(2) .unhealthyThresholdCount(3).build())
```

```
.deregistrationDelay(Duration.seconds(300)) .slowStart(Duration.seconds(60)) .build();
service.attachToApplicationTargetGroup(targetGroup); } } // Usage: one line per service new
ShopFlowFargateService(this, "ProductSvc", ShopFlowServiceProps.builder()
.serviceName("product-svc").cluster(cluster).vpc(vpc).ecsSg(ecsSg)
.repo(productSvcRepo).cpu(512).memoryMiB(1024) .desiredCount(6).minCount(2).maxCount(20)
.secrets(Map.of("DB_PASSWORD", Secret.fromSsmParameter(dbPasswordParam))) .build());
```

15-20 Secrets Management — SSM Parameter Store in CDK

ShopFlow never puts database passwords or API keys in CDK code or environment variables. All secrets are stored in AWS Systems Manager Parameter Store (SecureString type) and referenced in CDK — ECS fetches them at task startup.

```
// CDK: Reference SSM parameters (created separately, not in CDK code) // SecureString
parameters are encrypted with KMS and never appear in CDK template // Reference existing SSM
parameter in CDK IStringParameter auroraPassword =
StringParameter.fromSecureStringParameterAttributes( this, "AuroraPassword",
SecureStringParameterAttributes.builder() .parameterName("/shopflow/prod/aurora/password")
.version(1) // pin to version; update when password rotates .build()); // Pass to ECS container
as Secret (ECS fetches at task startup) taskDef.addContainer("app",
ContainerDefinitionOptions.builder() .secrets(Map.of( // SPRING_DATASOURCE_PASSWORD env var =
value from SSM "SPRING_DATASOURCE_PASSWORD", Secret.fromSsmParameter(auroraPassword),
"REDIS_AUTH_TOKEN",
Secret.fromSsmParameter(StringParameter.fromSecureStringParameterAttributes( this,
"RedisToken", SecureStringParameterAttributes.builder()
.parameterName("/shopflow/prod/redis/auth-token") .build()))))) .build()); // Grant ECS task
role permission to read the SSM parameters auroraPassword.grantRead(taskRole); // Result:
Docker container sees SPRING_DATASOURCE_PASSWORD as env var // Value is fetched from SSM at task
startup; never in CloudFormation template // Rotate the SSM value -> next ECS task restart picks
up new password automatically
```

15-21 CDK Pipelines — Self-Mutating Infrastructure Pipeline

CDK Pipelines is a higher-level CDK construct that creates a CodePipeline for deploying CDK apps. It is "self-mutating" — when you change the pipeline definition in CDK code, the pipeline updates itself on the next run.

```
// PipelineStack.java: CDK Pipeline that deploys ShopFlow public class PipelineStack extends
Stack { public PipelineStack(App app, String id, StackProps props) { super(app, id, props); //
Connect to GitHub repository CodePipelineSource source = CodePipelineSource.gitHub(
"shopflow-org/shopflow-cdk", "main", // deploy on every push to main
GitHubSourceOptions.builder() .authentication(SecretValue.secretsManager("github-token"))
.build()); // CDK Pipeline: synth step runs "cdk synth" CodePipeline pipeline =
CodePipeline.Builder.create(this, "Pipeline") .pipelineName("ShopFlow-CDK-Pipeline")
.synth(ShellStep.Builder.create("Synth") .input(source) .commands(List.of( "npm install -g
aws-cdk", "mvn -q package", "cdk synth")) .build()) .build(); // Deploy to staging first, then
prod (with manual approval gate) ShopFlowStage stagingStage = new ShopFlowStage(this,
"Staging", StageProps.builder()
.env(Environment.builder().account("111111111").region("us-east-1").build()) .build());
pipeline.addStage(stagingStage, AddStageOpts.builder()
.pre(List.of(ShellStep.Builder.create("RunTests") .commands(List.of("mvn test")).build()))
.build()); ShopFlowStage prodStage = new ShopFlowStage(this, "Prod", StageProps.builder()
.env(Environment.builder().account("222222222").region("us-east-1").build()) .build());
pipeline.addStage(prodStage, AddStageOpts.builder() .pre(List.of(new
ManualApprovalStep("ApproveProdDeploy"))) .build()); } }
```

15-22 Black Friday — Infrastructure Preparation with CDK

Before Black Friday, ShopFlow uses CDK to provision extra capacity. Instead of manually scaling resources, they update CDK context values and deploy — the pipeline scales everything safely and consistently.

```
// cdk.json: Black Friday context values (applied 1 week before) { "context": { "env": "prod",
// Normal values -> Black Friday values: "productSvcMinTasks": 6, // 6 -> 20 (pre-warm; don't
wait for auto-scale) "productSvcMaxTasks": 20, // 20 -> 60 "orderSvcMinTasks": 4, // 4 -> 15
"orderSvcMaxTasks": 15, // 15 -> 40 "aurorReaderInstances": 2, // 2 -> 4 (more read replicas for
product catalog) "elasticacheShardsCount": 3, // 3 -> 6 (more Redis shards for session data)
```

```
"dynamoOrdersWCU": 1000, // pay-per-request (automatic) -> keep as-is "albWarmupEnabled": true
// request ALB pre-warming via AWS Support } } // CDK auto-scaling config uses context values:
service.autoScaleTaskCount(EnableScalingProps.builder() .minCapacity((Integer)
node.tryGetContext("productSvcMinTasks")) .maxCapacity((Integer)
node.tryGetContext("productSvcMaxTasks")) .build()); // Deploy Black Friday config: // git
commit -m "BF2026: pre-warm capacity" // git push origin main // Pipeline triggers
automatically; stages deploy in order // Staging -> (auto-approved) -> Prod (manual approval
from SRE lead) // Total time: ~35 minutes from commit to fully scaled production environment
```

15-23 Chapter Summary — Infrastructure as Code with CDK

You now understand how ShopFlow uses CDK and CloudFormation to manage its entire AWS environment as code. Here is what each tool and concept gives you.

What You Learned	Key Takeaway
CloudFormation vs CDK relationship	CDK synthesizes CloudFormation templates from Java code. You write CDK; CloudFormation deplo
CDK constructs (L1/L2/L3)	L2 constructs (most CDK classes) have secure defaults. L3 patterns (ApplicationLoadBalancedFarga
Stack organization	One stack per lifecycle: Network, Data, Services, Traffic, Observability, Pipeline. Stacks declare dep
cdk diff before every deploy	cdk diff is non-negotiable before cdk deploy. A "Replace: true" on a database means DATA LOSS. A
Reusable constructs	ShopFlowFargateService construct gives every microservice consistent monitoring, security groups,
Secrets via SSM Parameter Store	Never put database passwords in CDK code. Store in SSM SecureString; reference in CDK; ECS fet
CDK Pipelines (self-mutating)	Push to main -> pipeline synths CDK -> deploys to staging -> manual approval -> deploys to prod. Pi
Black Friday readiness	Update context values in cdk.json (min tasks, max tasks, replicas) -> push -> pipeline scales everyth

15-24 CloudFormation Change Sets — Safe Production Updates

A Change Set is a preview of what CloudFormation will do before it actually does it. CDK uses Change Sets automatically under the hood. For production deployments, the ShopFlow team always reviews the Change Set before approving.

■ Review a Change Set Before Deploying
1. cdk diff outputs a summary — but for production, review the actual CloudFormation Change Set
2. CloudFormation -> Stacks -> ShopFlowServicesStack -> "Stack actions" -> "Create change set"
3. Upload the new template (from cdk synth output in cdk.out/)
4. Change Set type: "Update stack" -> Next -> Create change set
5. Wait ~30 seconds for Change Set to calculate
6. Review the Change Set table: Action (Add/Modify/Remove), Resource, Replacement (True/False)
7. If any row shows "Replacement: True" on a critical resource -> do not proceed; investigate
8. If all looks correct: "Execute change set" -> CloudFormation deploys
9. If Change Set has issues: "Delete change set" -> fix CDK code -> try again

Change Set Action	Meaning	ShopFlow Example	Safe?
Add	New resource will be created	New CloudWatch Alarm or new ECS Task Definition revision	Safe — additive
Modify (no replacement)	Resource property updated in-place	ECS service desired count 6 -> 8; ALB listener rule priority change	Safe — in-place update
Modify (conditional replacement)	May be replaced depending on current value	ECS task definition CPU 512 -> 1024 (creates new revision)	Safe — ECS rolling update

Change Set Action	Meaning	ShopFlow Example	Safe?
Modify (replacement: True)	Resource DELETED and recreated (data loss if stateful)	DynamoDB table partition key change — table dropped and recreated	DANGEROUS — stop and im
Remove	Resource deleted permanently	Old CloudWatch Alarm removed; old IAM role removed	Usually safe; verify nothing dep

15-25 CloudFormation Conditions — Environment-Specific Resources

CDK conditions let you create resources only in certain environments. ShopFlow uses conditions to create deletion protection in prod but not dev, and to enable Multi-AZ in prod but single-AZ in dev (cost savings).

```
// CDK: Conditions for environment-specific configuration public class DataStack extends Stack
{ public DataStack(App app, String id, DataStackProps props) { super(app, id, props); String env
= (String) this.getNode().tryGetContext("env"); boolean isProd = "prod".equals(env); // Aurora
cluster — Multi-AZ in prod, single-AZ in dev DatabaseCluster aurora =
DatabaseCluster.Builder.create(this, "Aurora") .engine(DatabaseClusterEngine.auroraPostgres(
AuroraPostgresClusterEngineProps.builder() .version(AuroraPostgresEngineVersion.VER_15_4)
.build())) .writer(ClusterInstance.provisioned("writer",
ProvisionedClusterInstanceProps.builder() .instanceType(isProd ?
InstanceType.of(InstanceClass.R6G, InstanceSize.XLARGE2) // prod: r6g.2xlarge :
InstanceType.of(InstanceClass.T3, InstanceSize.MEDIUM)) // dev: t3.medium .build()))
.readers(isProd ? List.of( ClusterInstance.provisioned("reader1", ...),
ClusterInstance.provisioned("reader2", ...)) // 2 read replicas in prod : List.of()) // no read
replicas in dev .storageEncrypted(true) // always encrypt .deletionProtection(isProd) //
deletion protection only in prod .removalPolicy(isProd ? RemovalPolicy.RETAIN // prod: retain
DB on stack delete : RemovalPolicy.DESTROY) // dev: delete DB on cdk destroy .build(); } } //
Deploy to dev: cdk deploy -c env=dev (small instances, no replicas, deletable) // Deploy to
prod: cdk deploy -c env=prod (large instances, 2 replicas, protected) // Same CDK code handles
both environments
```

15-26 CloudFormation Outputs and Cross-Stack Exports

CloudFormation Outputs expose values from a stack (like VPC ID or ALB DNS name) so they can be used by other stacks or viewed in the console. When CDK passes a resource from one stack to another, it automatically creates Outputs and Imports.

```
// CDK: Explicit Output (shows value in CloudFormation console) new CfnOutput(this,
"AlbDnsName", CfnOutputProps.builder() .value(alb.getLoadBalancerDnsName()) .description("ALB
DNS name — point Route 53 ALIAS record here") .exportName("ShopFlowServicesStack-AlbDnsName")
.build()); new CfnOutput(this, "EcrProductSvcUri", CfnOutputProps.builder()
.value(productSvcRepo.getRepositoryUri()) .description("ECR URI for product-svc Docker images")
.exportName("ShopFlowServicesStack-ProductSvcEcrUri") .build()); // View outputs in console: //
CloudFormation -> Stacks -> ShopFlowServicesStack -> Outputs tab // Also accessible via CLI: //
aws cloudformation describe-stacks --stack-name ShopFlowServicesStack // --query
"Stacks[0].Outputs" // Cross-stack import (automatic when you pass CDK objects between stacks):
// NetworkStack exports VpcId as "ShopFlowNetworkStack-ShopFlowVPC-VpcId" // DataStack
auto-imports it when you reference network.getVpc() // CDK generates: { "Fn::ImportValue":
"ShopFlowNetworkStack-ShopFlowVPC-VpcId" }
```

15-27 Troubleshooting CDK and CloudFormation Errors

These are the most common CDK and CloudFormation errors ShopFlow engineers encounter and how to fix them.

Error	Cause	How to Fix
Error: This stack uses assets, so the toolkit stack must be deployed	CDK bootstrap not run in this account/region	Run cdk bootstrap aws://ACCOUNT-ID/REGION once per acco
ROLLBACK_COMPLETE — stack cannot be updated	Stack in failed state after a previous deployment failure	Delete the stack (cdk destroy or CloudFormation console) and re
Resource already exists with different properties	A resource with the same name exists in AWS but not in this CloudFormation stack	Import the resource into CDK or use ... (created manually
Export ShopFlowNetworkStack-Vpc... cannot be deleted because it is in use	Another stack imports this export	Deploy the importing stack first (to remove the import),
Template format error: Every Default member must be a string	Java integer passed where CloudFormation expects a string	Ensure CDK number-to-string conversion; often a Cfn* (L1) prop

Error	Cause	How to Fix
Circular dependency between stacks	Stack A imports from Stack B which imports from Stack A	Restructure: extract the shared resource to a third stack that both

15-28 Chapter Q&A — Infrastructure as Code

Question	Answer
Should I use CDK or Terraform for AWS?	Both are valid choices. CDK is better if your team writes Java/Python/TypeScript already and pr
Can I mix CDK-managed and manually-created resources?	Yes, but with caution. Use CDK fromLookup or fromArn to reference manually-created resource
How do I handle database schema migrations with CDK?	CDK manages infrastructure (create the Aurora cluster), not schema (CREATE TABLE). Use F
What is the difference between cdk synth and cdk deploy?	cdk synth generates the CloudFormation template files in cdk.out/ — does not deploy anything.

15-29 Multi-Environment Configuration — Dev to Prod

ShopFlow maintains four environments: local, dev, staging, and prod. All use the same CDK code with different context values. The table below shows how resource sizes differ across environments.

Resource	Local (laptop)	Dev (AWS)	Staging (AWS)	Prod (AWS)
Aurora instance	H2 in-memory (no AWS cost)	db.t3.medium x1 (no replicas)	db.r6g.large x1 + 1 reader	db.r6g.2xlarge x1 + 2 readers
ECS task count	1 (Docker Compose)	1 task per service	2 tasks per service	6 tasks per service (min)
ECS task size	N/A	256 CPU / 512 MB	512 CPU / 1024 MB	512 CPU / 1024 MB
ElastiCache Redis	Redis Docker container	cache.t3.micro x1	cache.r6g.large x2 (1 replica)	cache.r6g.large x6 (cluster mode, 3 sh
DynamoDB capacity	DynamoDB Local (Docker)	On-demand	On-demand	On-demand (BF: pre-provisioned 200
CloudFront	N/A (localhost)	Disabled	Enabled	Enabled + WAF
Monthly AWS cost	~$0	~$80	~$350	~$3,200

15-30 CDK Security Best Practices — Secure Defaults

CDK L2 constructs have secure defaults in most cases, but you must be intentional about a few security settings. Here is what ShopFlow checks in code review for every CDK change.

Resource	Insecure Default to Avoid	Secure ShopFlow Configuration
S3 Bucket	Public access can be enabled (old buckets)	BlockPublicAccess.BLOCK_ALL + versioned(true) + encryption(BucketEncry
RDS Aurora	Not encrypted by default in some CDK versions	storageEncrypted(true) + deletionProtection(true) in prod + snapshotIdentifie
DynamoDB	No point-in-time recovery by default	pointInTimeRecovery(true) always; tableClass: STANDARD; billing: ON_DE
Lambda	Broad execution role if using addToRolePolicy carelessly	Principle of least privilege: only grant the specific actions and resources the f
ECS Task Role	Empty role (no permissions)	Explicitly grant only: SSM GetParameter for secrets, X-Ray PutTraceSegme
VPC	No flow logs by default	Enable VPC Flow Logs with REJECT traffic to S3 or CloudWatch for security
Secrets	Hardcoded in environment variables	Always use SSM Parameter Store SecureString or Secrets Manager; never

15-31 Well-Architected Review — Infrastructure as Code

The AWS Well-Architected Framework has specific guidance for operational excellence around infrastructure management. ShopFlow evaluates its CDK practices against these standards.

Pillar	Requirement	ShopFlow CDK Status
Operational Excellence	Use version control; deploy via automated pipeline; detect and remediate drift	CDK in GitHub; CDK Pipelines deploys every push to main;
Security	Apply security at every layer; enforce least privilege; audit changes	CDK Aspects enforce encryption + tagging; ECS task roles a
Reliability	Test infrastructure changes in lower environments first; have rollback	Staging -> manual approval -> prod pipeline; CloudFormatio
Performance Efficiency	Right-size resources per environment; use managed services	Context-driven sizing (dev=small, prod=large); all ShopFlow
Cost Optimization	Tag resources for cost allocation; decommission unused resources	CDK Aspect enforces Project/Environment tags on every res

15-32 CDK Quick Reference — Commands and Patterns

A concise reference card for the CDK concepts and commands used in this chapter.

Task	Command or Pattern
Install CDK CLI	npm install -g aws-cdk (requires Node.js 18+)
Bootstrap new account/region	cdk bootstrap aws://ACCOUNT-ID/us-east-1
Create new Java CDK project	cdk init app --language java
Compile + synth CloudFormation	cdk synth (outputs to cdk.out/)
Preview changes before deploy	cdk diff (shows add/modify/remove; check for Replace: true)
Deploy one stack	cdk deploy ShopFlowServicesStack
Deploy all stacks in order	cdk deploy --all
Delete a stack (irreversible)	cdk destroy ShopFlowServicesStack
List all stacks	cdk list
Pass context value on command line	cdk deploy -c env=prod -c minTasks=20
CDK unit test command	mvn test (JUnit 5 + CDK Template assertions)
Import existing resource	Vpc.fromLookup() or fromArn() or fromAttributes()
Enable Container Insights	Cluster.Builder...containerInsights(true)
Enforce tags on all resources	Aspects.of(app).add(new TaggingAspect())
Reference SSM secret in ECS	Secret.fromSsmParameter(StringParameter.fromSecureStringParameterAttributes(...))
CDK Construct	**What It Creates**
Vpc.Builder with SubnetConfiguration	VPC + subnets + route tables + internet gateway + NAT gateways
ApplicationLoadBalancer.Builder	ALB (no listeners; add with addListener)
ApplicationLoadBalancedFargateService.Builder (L3)	ECS task + Fargate service + ALB + listener + target group in one
DatabaseCluster.Builder (Aurora)	Aurora cluster + writer instance + reader instances + subnet group + parameter group
Table.Builder (DynamoDB)	DynamoDB table + optional GSIs + auto-scaling
CachingCluster.Builder (ElastiCache)	ElastiCache cluster + subnet group + security group
CodePipeline.Builder (CDK Pipelines)	CodePipeline + CodeBuild + S3 bucket + IAM roles (self-mutating)

15-33 CDK Escape Hatches — When CDK Does Not Cover a Feature

AWS releases new features daily. CDK L2 constructs sometimes lag behind. CDK provides "escape hatches" to access the underlying CloudFormation resource (L1) and set any property directly — without waiting for CDK to add L2 support.

```
// Escape hatch: set a property not yet supported by CDK L2 // Example 1: Aurora Serverless v2
scaling configuration // CDK L2 ServerlessCluster does not support all v2 properties yet // Use
escape hatch to set via L1 CfnDBCluster DatabaseCluster auroraCluster =
DatabaseCluster.Builder.create(this, "Aurora")
.engine(DatabaseClusterEngine.auroraPostgres(...)) .serverlessV2MinCapacity(0.5)
.serverlessV2MaxCapacity(64) .build(); // Access underlying L1 resource CfnDBCluster cfnCluster
= (CfnDBCluster) auroraCluster.getNode() .findChild("Resource"); // "Resource" is the default
L1 child name // Set property not available in L2
cfnCluster.setPerformanceInsightsEnabled(true);
cfnCluster.setPerformanceInsightsRetentionPeriod(731); // 731 days = 2 years
cfnCluster.setMonitoringInterval(60); // Enhanced Monitoring every 60s // Example 2: Add a
custom CloudFormation resource for a feature CDK doesn't support // AwsCustomResource lets you
call any AWS API as a CloudFormation resource AwsCustomResource customResource =
AwsCustomResource.Builder.create(this, "EnableFeatureX") .onCreate(AwsSdkCall.builder()
.service("SomeAWSService") .action("enableFeature") .parameters(Map.of("resourceArn",
cluster.getClusterArn())) .physicalResourceId(PhysicalResourceId.of("feature-x-enabled"))
.build()) .onDelete(AwsSdkCall.builder() .service("SomeAWSService") .action("disableFeature")
.parameters(Map.of("resourceArn", cluster.getClusterArn())) .build())
.policy(AwsCustomResourcePolicy.fromSdkCalls(
SdkCallsPolicyOptions.builder().resources(List.of(cluster.getClusterArn())).build())) .build();
// Rule: use L2 -> L3 when possible. Only drop to escape hatch when L2 is missing a needed
property. // Document why you used an escape hatch in a code comment.
```

CDK Level	Use When	ShopFlow Example
L3 Pattern	A fully-opinionated solution meets your needs	ApplicationLoadBalancedFargateService for quick serv
L2 Construct	You need standard configuration with some customization	FargateService with custom security groups and auto-s
L1 Escape Hatch	L2 missing a specific property you need	Aurora performance insights retention; custom WAF rul
AwsCustomResource	No CloudFormation support at all for a feature	Enable a feature via AWS API that has no CloudForma